

SKIVE on gpt-oss-20b for AIME 2024: everything we did, change by change

This document covers only the AIME 2024 work: how the benchmark was run, what gpt-oss required that other models did not, every code change made for it and why, what each change did to the numbers, and where the work stands. All numbers are measured on one NVIDIA L40S (48 GB) between 2026-09-22 and 2026-09-23.

1. The task and how it was measured

AIME 2024 is 30 competition math problems with integer answers. gpt-oss-20b solves them by generating a long reasoning trace (about 7,000 tokens per attempt at medium reasoning effort) and then a short final answer. The KV cache of an AIME request is therefore almost entirely the model's own reasoning; the prompt is about 120 tokens.

Protocol. Every run uses all 30 problems. The first runs used 4 samples per problem (temperature 0.6, top_p 0.95, up to 16,384 output tokens); the final sweep used 8, which means 240 concurrent requests per run. Accuracy is the pass rate: the fraction of samples whose boxed integer equals the reference, averaged over problems. With 240 samples one sample is 0.4 points, so differences below about 2.5 points are noise. The TriAttention paper uses 8 samples at 32k tokens and reports 69.2 for FullKV on this model; our FullKV lands at 68.8 to 74.2 across runs, consistent with that.

What is recorded per run. Besides accuracy: cross% (share of requests that evicted at least one block), eviction rate (cache tokens evicted per generated token), evicted blocks per request, KV saved (evicted tokens over total tokens), rep4g (repeated 4-gram fraction, a looping detector), ttft, tpot and end-to-end latency from vLLM's own per-request stats, wall time, generated tokens per sample, batch throughput, and the engine's maximum concurrency. Every generated answer is written to a CSV, one row per sample.

2. Why gpt-oss is different, and what had to change before AIME could run at all

gpt-oss-20b differs from the models SKIVE was validated on (Qwen, Llama, DeepSeek-Distill) in five ways. Each one broke an assumption in the original integration.

gpt-oss fact	consequence	change made
Alternating attention: even layers use a 128-token sliding window, odd layers full attention	vLLM creates two KV-cache groups, each with its own block table and manager; group 0 is not guaranteed to be the full-attention one. The original code hard-coded index 0 and would have evicted from the sliding-window table.	<code>_skive_full_group</code> in <code>kv_evict/integration.py</code> inspects the KV-cache configuration, picks the group whose spec is true full attention, evicts, scores and reclaims only there, and refuses to evict if the group cannot be resolved. Freed blocks carry their group id. On the box the full group resolved as group 1, confirming the fix was needed.
Learned attention sinks: every head has a sink logit inside the softmax	An importance score computed with a plain softmax uses a different distribution than the model.	<code>_softmax_with_sink</code> appends the per-head sink logit before the softmax and drops it after.
Grouped-query attention, 64 query heads on 8 KV heads	Summing raw per-head scores lets a few large-magnitude heads dominate the block ranking.	<code>SKIVE_HEAD_AGG=zmax</code> : z-score each head over its tokens, then take the max across heads (TriAttention's aggregation).
Sinks are only served by three vLLM backends: FlashAttention-3 (Hopper only), FlashInfer TRT-LLM (SM100 only), or the Triton unified backend	On the L40S (compute capability 8.9) vLLM falls through to the Triton backend. SKIVE's query capture and sparse-gather hooks existed only in the FlashAttention file, so on this GPU <code>value_attention</code> would have silently degraded to <code>vk_ratio</code> and evicted blocks would still have been read.	Patch edit K in <code>stage4/patch_vllm.py</code> adds the same two hooks to <code>triton_attn.py</code> : <code>layer._skive_q = query.detach()</code> at the top of the Triton forward, and the block-table compaction in the Triton metadata builder, gated to the full-attention group and skipped only under full CUDA-graph capture. The runners force the backend (<code>SKIVE_ATTN_BACKEND=auto</code> : FlashAttention on Hopper, Triton elsewhere).
Harmony output format with a reasoning channel	The raw completion contains the analysis channel and control tokens; scoring it as the answer is wrong, and <code>reasoning_effort</code> must be passed through the chat template.	The harness decodes with special tokens kept, extracts the <code>final</code> channel only, and passes <code>reasoning_effort</code> as a chat-template argument (system-prompt fallback).

Two engine-level facts also surfaced on this box. First, the MXFP4 expert weights run on the Marlin kernels on Ada GPUs (the FlashInfer variants are SM90/SM100 only); this worked without changes. Second, reading SKIVE's per-request eviction counter across the multiprocess engine boundary needs `VLLM_ALLOW_INSECURE_SERIALIZATION=1`, otherwise vLLM's msgpack encoder refuses to serialize the callable; without that flag cross% and eviction rate would have been blank.

3. The first AIME result and its diagnosis

The first full run (4 samples, budget 2048 tokens) gave FullKV 73.3, vk_ratio 47.5, value_attention 49.2. Two things were wrong: accuracy fell 24 points, far more than on summarization at a similar KV saving, and decode got slower under eviction (tpot 50 to 60 ms) although the cache was smaller.

Accuracy. SKIVE protects the first `num_sink_blocks` blocks (2 blocks, 32 tokens) and the last `num_local_blocks` (8 blocks, 128 tokens) of every request. The AIME prompt spans about 8 blocks. So once a trace exceeded 2048 tokens, three quarters of the problem statement were ordinary eviction candidates, and value_attention, which scores blocks by what the current decode token attends to, does not always rank the question highly mid-derivation. The traces showed the symptom: evicted runs generated 21 to 37 percent more tokens per sample, re-deriving what they had lost, with more looping.

Speed. The tpot increase was suspected to be forced eager mode rather than eviction work: sparse-gather rewrites the block table every step, which is not allowed inside a captured CUDA graph, so the runs used `enforce_eager=True`. On the gsm8k throughput sweep CUDA graphs alone had been worth 44 percent on decode-bound loads.

4. Code changes made for AIME, and why

4.1 `kv_evict/integration.py`

- **Prompt protection, `SKIVE_PROTECT_PROMPT=1`.** In `evict_request_blocks`, the sink count becomes `max(num_sink_blocks, ceil(prompt_tokens / block_size) + 1)`, with the prompt length read per request from vLLM's input batch (`input_batch.num_prompt_tokens`). Why: the problem statement must never be evicted on a generation-heavy task. Off by default because on long-prompt tasks (HotpotQA) the prompt is exactly what must be evicted. Effect: 49.2 to 65.0 at the same 77 percent KV saved.
- **Trajectory scoring, `SKIVE_QHIST=N`.** `_skive_capture_qhist` runs after every N-th decode step inside `skive_post_step` and stores, per scored layer, the query rows of all requests (one `index_select` per layer, kept in bf16, in a bounded deque on the runner). At eviction time `_batch_score_val` looks up each over-budget request in that history, stacks its recent queries with the current one, and `_score_multi_query` computes the sink-aware attention for every query in the window and averages it before the value weighting. Rows are chunked so the `[rows, window, heads, tokens]` logits stay under about 256 MB. Why: a single query is a noisy estimate of what the next few hundred tokens will read; SnapKV, R-KV and TriAttention all score with an observation window. Effect on AIME: none (63.3 vs 63.8), and tpot rose from 73 to 104 ms. Left off by default.
- **Redundancy penalty, `SKIVE_REDUNDANCY=lambda`.** `_redundancy` computes each block's mean key, its cosine similarity to every other real block of the same request, and takes the maximum; `_zscore_rows` standardizes importance and redundancy per request and the final score is `z(importance) - lambda * z(redundancy)`. Why: reasoning traces restate themselves, and an importance-only score can keep five restatements while dropping the one block with a needed intermediate result. Effect on AIME: within noise (64.6 vs 63.8). Left off by default.

- **FP8 cache decoding, `_kv_blocks_f32`.** vLLM stores an fp8 KV cache as uint8; the scoring code now views such blocks as float8, converts to float32 and applies the layer's per-tensor scales, for both the static vk_ratio score and the value_attention path. Why: to make an FP8 cache, which halves bytes per token, usable together with eviction.
- **Debug path.** With `SKIVE_DBG2=1` the batched scorer prints its traceback instead of silently falling back to vk_ratio, so a failure in the new code cannot masquerade as a result.

4.2 `stage6/gptoss_campaign.py` (the evaluation harness)

- **AIME loader and scoring.** Loads all 30 problems, prompts for a boxed integer, samples `NSAMP` completions per problem, extracts the last boxed number from the harmony `final` channel, and averages correctness over samples. Every sample's text is stored, so the CSV has one row per sample.
- **Reasoning effort.** `REASONING=medium` for AIME (low for the other datasets) passed through the chat template; the earlier `low` setting would understate the model on competition math.
- **Piecewise CUDA graphs, `CUDAGRAPH=piecewise`.** Sets `enforce_eager=False` with `CompilationConfig(cuda_graph_mode=PIECEWISE)`. Why: attention stays eager, so SKIVE's block-table edits and compaction remain legal, while the MoE and MLP pieces that dominate this model get graph replay. Effect: tpot 64 to 51 ms, equal to FullKV eager; the evicting run became the fastest run overall.
- **FP8 cache and block size knobs, `KV_DTYPE`, `BLOCK_SIZE`.** Budgets are given in tokens and converted to blocks with the chosen block size, so 2048 tokens means 128 blocks at size 16 and 32 blocks at size 64.
- **Variants, `--tag`.** Result files and row labels carry a suffix (for example `va@2048+pp_132`), and every run records the knob values it ran with, so the tables are self-describing.
- **Eviction counting.** The per-request eviction counter is snapshotted after the warm-up request and again after the batch; only keys that appeared in between are counted. Why: the worker's request ids do not match the frontend's, and the warm-up request itself evicts, which had produced a cross% above 100 in the first attempt. From those counts the harness derives cross%, eviction rate (evicted tokens per generated token), evicted blocks per request, and KV saved.
- **CSV export, `csv --dataset aime24`.** Columns: config, budget, item, sample, question, reference, gold, answer, score, generated tokens.

4.3 Other files

- `stage4/patch_vllm.py`: edit K (Triton hooks), an `--only` filter to apply a subset of edits, and a no-backup mode for git trees.
- `stage6/run_aime_sweep.sh`, `run_best_sweep.sh`, `run_best_followup.sh`: the sweep drivers, resumable (a finished result file is skipped), each run its own process so `SKIVE_METRIC` and the CUDA-graph mode are clean.
- `tests/test_trajectory_scoring.py`: the multi-query scorer equals the single-query path for a window of one, averages over the window, respects sinks, chunks rows; the redundancy term flags duplicate blocks and ignores padding; fp8 blocks decode and scale correctly. `tests/test_triton_hooks.py`: edit K applies once, compiles, compacts decode rows only, and captures the query only for query-dependent metrics. 28 tests pass on the box.

5. Results: every AIME configuration

Final sweep, 8 samples per problem, 240 concurrent requests, piecewise CUDA graphs everywhere. Reference row is FullKV bf16 with graphs (68.8). Eviction rows use value_attention with prompt protection and a 32-block local window unless stated. traj = 16-query trajectory window, red = redundancy penalty, blk64 = 64-token blocks, fp8 = FP8 KV cache. Deltas vs the reference row.

Config	cross%	evict rate	evict blk/req	acc (dAcc)	rep4g	tpot (ms)	e2e (s)	wall (s)	gen tok / sample	batch tok/s	KV saved	max conc.
FullKV cg	-	-	-	68.8	0.035	87	550	788	7829	2384	-	32.9
FullKV cg fp8	-	-	-	71.7 (+2.9)	0.040	63 (-28%)	407 (-26%)	603 (-23%)	7765	3089 (+30%)	-	65.0
va @2048	89%	0.78	414	63.8 (-5.0)	0.049	73 (-16%)	486 (-12%)	656 (-17%)	8431	3085 (+29%)	78%	32.9
va @2048 red	89%	0.79	437	64.6 (-4.2)	0.057	71 (-18%)	500 (-9%)	674 (-14%)	8815	3138 (+32%)	79%	32.9
va @2048 traj	90%	0.79	430	63.3 (-5.5)	0.044	104 (+20%)	557 (+1%)	739 (-6%)	8710	2828 (+19%)	79%	32.9
va @2048 traj red	90%	0.80	448	62.9 (-5.9)	0.055	97 (+11%)	580 (+5%)	754 (-4%)	8987	2859 (+20%)	80%	32.9
va @2048 blk64	87%	0.79	108	60.0 (-8.8)	0.064	58 (-33%)	418 (-24%)	572 (-27%)	8707	3653 (+53%)	79%	32.9
va @2048 traj blk64	88%	0.80	110	63.8 (-5.0)	0.046	81 (-7%)	473 (-14%)	644 (-18%)	8835	3294 (+38%)	80%	32.9
va @4096	64%	0.56	272	72.1 (+3.3)	0.026	81 (-7%)	507 (-8%)	721 (-9%)	7705	2565 (+8%)	56%	32.9
va @4096 traj	63%	0.58	292	70.4 (+1.6)	0.038	91 (+5%)	584 (+6%)	821 (+4%)	8023	2346 (-2%)	58%	32.9
va @4096 fp8	65%	0.61	333	61.2 (-7.6)	0.056	78 (-10%)	523 (-5%)	692 (-12%)	8696	3015 (+26%)	61%	65.0
va @4096 traj fp8	64%	0.60	319	68.3 (-0.5)	0.048	86 (-1%)	545 (-1%)	739 (-6%)	8464	2751 (+15%)	60%	65.0

Earlier 4-sample rows for the record: FullKV eager 73.3; vk @2048 47.5; va @2048 49.2; va @2048 with prompt protection 65.0; plus local 32: 66.7; plus eviction every 64 steps: 63.3; plus piecewise graphs: 64.2 (tpot 51 vs FullKV eager 50); va @4096 without prompt protection 68.3.

6. What each change did, and why, in the model's terms

- **Prompt protection was the accuracy fix (+16 points at 2048).** The question is the one span the trace refers back to throughout; once it is guaranteed to stay, the remaining loss comes only from evicted intermediate steps.
- **Budget 4096 is lossless on AIME (72.1 vs 68.8 to 71.7 FullKV) at 56 percent of KV saved.** Traces average 7k tokens; a 4096 budget plus prompt protection keeps most of the active derivation. At 2048 the model must re-derive (8 to 15 percent more tokens per sample) and loses 4 to 6 points.
- **Piecewise CUDA graphs removed the decode penalty.** The 20B MoE's step time is dominated by expert and MLP kernels, not attention, so shrinking the cache buys little while eager mode costs a lot. With graphs, eviction at 2048 runs 6 to 27 percent faster than FullKV in wall time at 240 concurrent traces, because FullKV is then memory-bound and queues while eviction keeps 20 percent of the cache.
- **Trajectory scoring and redundancy did not help on AIME.** The blocks that matter in a derivation are recent and referenced repeatedly; the single current query already ranks them well once the prompt is protected, and averaging over 16 queries dilutes rather than sharpens that signal while costing scoring time. On HotpotQA, where the needed passages are far from the query, the same two changes added 2 to 5 F1, so they stay as opt-in knobs.
- **64-token blocks trade accuracy for speed.** Four times fewer evictions per request (108 vs 414) gives the lowest tpot and wall of any run, but each eviction drops a whole reasoning step, costing 9 points. Useful when speed matters more than the last points.
- **FP8 KV cache is a clear win on its own and unverified with eviction.** FullKV in FP8 scores 71.7, doubles the pool to 65 sequences and cuts wall time 23 percent. Combined with eviction the two runs disagree (61.2 and 68.3), which points at quantization noise in the block scores; it needs more samples before a conclusion.

7. Recommended configuration and what is left

For gpt-oss reasoning workloads: `value_attention`, `SKIVE_PROTECT_PROMPT=1`, local window 32 blocks, piecewise CUDA graphs, budget 4096 tokens (lossless on AIME, 56 percent saved) or 2048 (about 64 percent, 78 percent saved, fastest under load). FP8 KV cache for FullKV is recommended today; FP8 plus eviction is not yet.

Still open: a budget expressed as a fraction of the trace rather than a fixed token count; token-level repacking inside blocks so an eviction does not drop a whole reasoning step (the mechanism behind the block-64 loss); and re-running the FP8 plus eviction rows at 8 samples to settle whether the scores need a different treatment of quantized values. The answer files, per-run JSON and tables for every row above are in `gptoss_results3/`.